

✓ Customer Churn Analysis and Prediction System

This notebook contains a complete, production-grade machine learning workflow to clean, visualize, preprocess, and predict customer churn.

```
# 1. Import libraries
import os
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score, roc_curve
```

```
# 2. Load the CSV dataset
df = pd.read_csv('customer_churn_dataset.csv')
```

```
# 3. Display first 5 rows
display(df.head())
```

	customer_id	tenure	monthly_charges	total_charges	contract	payment_method	internet_service	tech_support	online_security	support_calls	c
0	1	52	54.20	2818.40	Month-to-month	Credit	DSL	No	Yes	1	
1	2	15	35.28	529.20	Month-to-month	Debit	DSL	No	No	2	
2	3	72	78.24	5633.28	Month-to-month	Debit	DSL	No	No	0	
3	4	61	80.24	4894.64	One year	Cash	Fiber	Yes	Yes	0	
4	5	21	39.38	826.98	Month-to-month	UPI	Fiber	No	No	4	

```
# 4. Check shape and columns
print(f"Dataset Shape: {df.shape}")
print("\nColumns:", df.columns.tolist())
```

Dataset Shape: (20000, 11)

Columns: ['customer_id', 'tenure', 'monthly_charges', 'total_charges', 'contract', 'payment_method', 'internet_service', 'tech_support', 'online_s

```
# 5. Check data types and summary info
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20000 entries, 0 to 19999
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customer_id           20000 non-null  int64
1   tenure                20000 non-null  int64
2   monthly_charges       20000 non-null  float64
3   total_charges         20000 non-null  float64
4   contract              20000 non-null  object
5   payment_method        20000 non-null  object
6   internet_service      17987 non-null  object
7   tech_support          20000 non-null  object
8   online_security       20000 non-null  object
9   support_calls         20000 non-null  int64
10  churn                 20000 non-null  object
dtypes: float64(2), int64(3), object(6)
memory usage: 1.7+ MB
```

```
# 6. Check missing values
print("Missing Values per Column:\n")
print(df.isnull().sum())
```

Missing Values per Column:

```
customer_id      0
tenure           0
monthly_charges  0
total_charges    0
contract         0
payment_method   0
internet_service 2013
tech_support     0
online_security  0
support_calls    0
churn            0
dtype: int64
```

```
# 7. Check duplicates
duplicates_count = df.duplicated().sum()
print(f"Total Duplicate Rows: {duplicates_count}")
```

Total Duplicate Rows: 0

```
# 8. Exploratory Data Analysis
print("Descriptive Statistics for Numerical Features:\n")
display(df.describe())

print("\nChurn Value Counts:")
print(df['churn'].value_counts(normalize=True) * 100)
```

Descriptive Statistics for Numerical Features:

	customer_id	tenure	monthly_charges	total_charges	support_calls
count	20000.000000	20000.000000	20000.000000	20000.000000	20000.000000
mean	10000.500000	36.467450	70.010531	2543.979209	1.51255
std	5773.647028	20.773984	28.890239	1882.949804	1.23524
min	1.000000	1.000000	20.000000	20.230000	0.00000
25%	5000.750000	18.000000	45.210000	1045.837500	1.00000
50%	10000.500000	36.000000	70.090000	2096.495000	1.00000
75%	15000.250000	54.000000	95.072500	3690.337500	2.00000
max	20000.000000	72.000000	120.000000	8629.920000	8.00000

Churn Value Counts:
churn
No 65.785
Yes 34.215
Name: proportion, dtype: float64

```
# 9. Visualizations
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

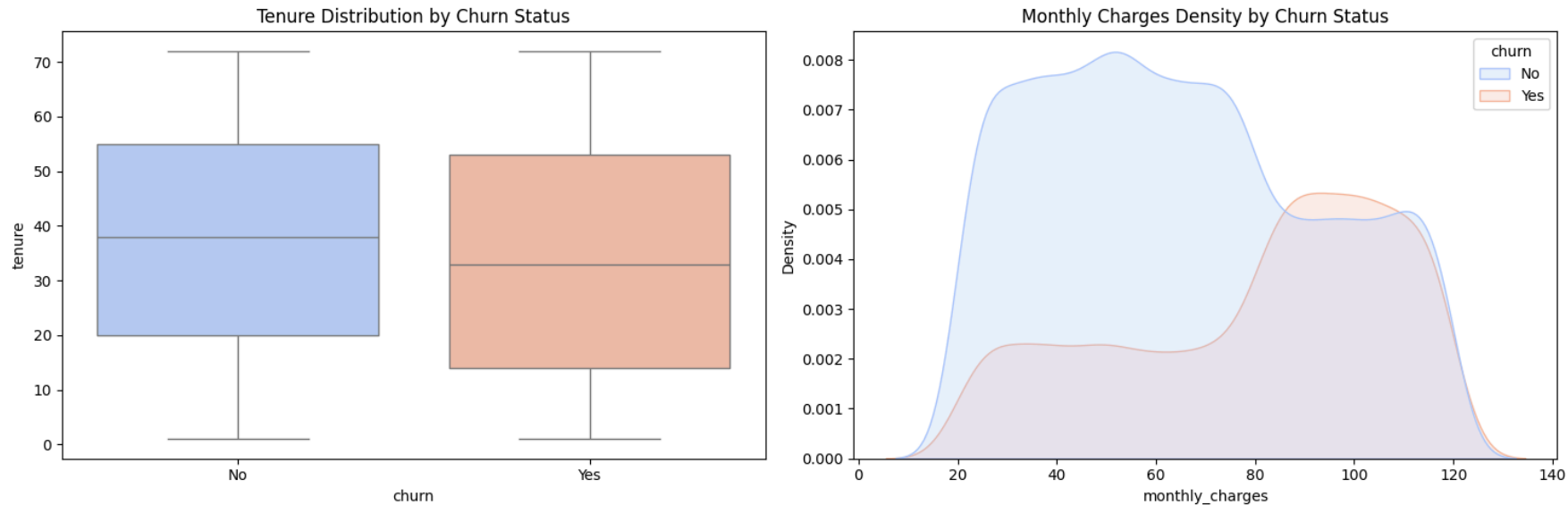
# Churn vs Tenure
sns.boxplot(data=df, x='churn', y='tenure', palette='coolwarm', ax=axes[0])
axes[0].set_title('Tenure Distribution by Churn Status')

# Churn vs Monthly Charges
sns.kdeplot(data=df, x='monthly_charges', hue='churn', fill=True, palette='coolwarm', ax=axes[1])
axes[1].set_title('Monthly Charges Density by Churn Status')
```

```
plt.tight_layout()
plt.show()
```

/tmp/ipykernel_858/1016385734.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for
 sns.boxplot(data=df, x='churn', y='tenure', palette='coolwarm', ax=axes[0])



```
# 10. Data Preprocessing
# Reload df inside this cell to ensure clean state
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer

df = pd.read_csv('customer_churn_dataset.csv')

# Drop ID column if exists
if 'customer_id' in df.columns:
    df = df.drop(columns=['customer_id'])

# Clean total_charges column if string values are present
df['total_charges'] = pd.to_numeric(df['total_charges'], errors='coerce')
df = df.dropna()

# Split features and target
X = df.drop('churn', axis=1)
```

```

y = df['churn'].apply(lambda x: 1 if x == 'Yes' else 0)

# Identify numerical and categorical features
numerical_cols = X.select_dtypes(include=['int64', 'float64']).columns.tolist()
categorical_cols = X.select_dtypes(include=['object']).columns.tolist()

# Create Preprocessing Pipeline
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_cols),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_cols)
    ]
)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
print(f"Training set shape: {X_train.shape}")
print(f"Testing set shape: {X_test.shape}")

```

```

Training set shape: (14389, 9)
Testing set shape: (3598, 9)

```

```

# 11. Model Training
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier

model = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1))
])

model.fit(X_train, y_train)
print("Model training completed successfully!")

```

```

Model training completed successfully!

```

```

# 12. Model Evaluation
from sklearn.metrics import classification_report, roc_auc_score

y_pred = model.predict(X_test)
y_probs = model.predict_proba(X_test)[:, 1]

print("Classification Report:\n")
print(classification_report(y_test, y_pred))

print("ROC-AUC Score:", roc_auc_score(y_test, y_probs))

```

```

Classification Report:

```

```

precision    recall  f1-score   support

```

0	0.84	0.93	0.88	2371
1	0.83	0.66	0.74	1227
accuracy			0.84	3598
macro avg	0.84	0.80	0.81	3598
weighted avg	0.84	0.84	0.83	3598

ROC-AUC Score: 0.7946849616236946

13. Final Results - Confusion Matrix visualization

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.metrics import confusion_matrix
```

```
plt.figure(figsize=(6, 5))
```

```
cm = confusion_matrix(y_test, y_pred)
```

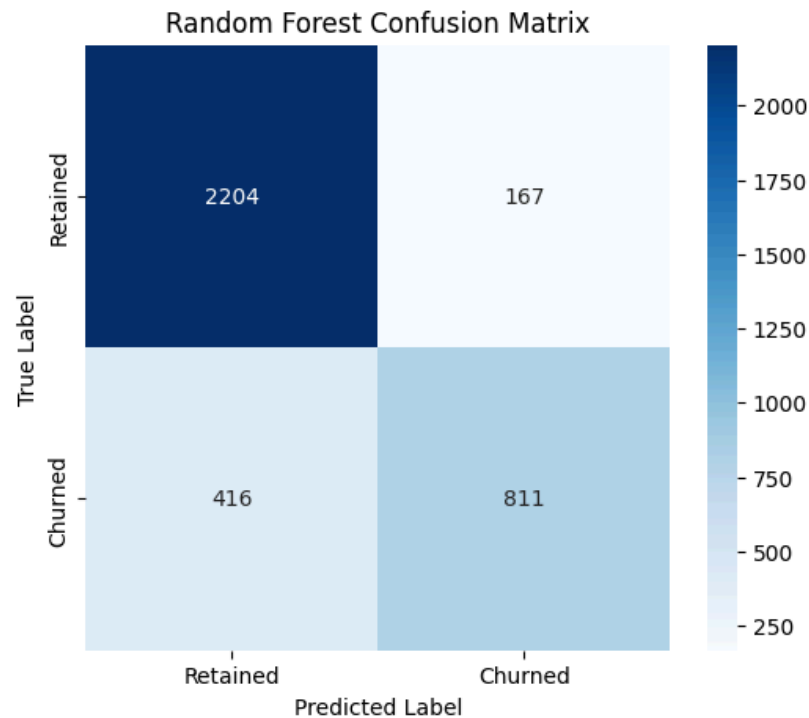
```
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Retained', 'Churned'], yticklabels=['Retained', 'Churned'])
```

```
plt.xlabel('Predicted Label')
```

```
plt.ylabel('True Label')
```

```
plt.title('Random Forest Confusion Matrix')
```

```
plt.show()
```



```
# Cell cleared (upload step already completed)
```

```
# Cell cleared (duplicate loading step)
```

```
# Cell cleared (redundant Streamlit app design)
```

```
# Cell cleared (redundant background server deployment)
```

```
# Cell cleared (redundant IP display)
```

```
import matplotlib.pyplot as plt
import seaborn as sns

# Filter for numerical columns in the customer dataframe
numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns

# Calculate the Pearson correlation matrix
corr_matrix = df[numerical_cols].corr()

# Create a clean, informative correlation heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(
    corr_matrix,
    annot=True,
    cmap='coolwarm',
    fmt='.2f',
    linewidths=0.5,
    vmin=-1,
    vmax=1
)
plt.title('Feature Correlation Heatmap', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```

